

Simulación — Playa de Estacionamiento

Documentación técnica y educativa · TP5 · Grupo 13

Este documento explica **qué hace el programa, cómo está armado**, las **decisiones de diseño** que tomamos (con su porqué) y **para qué sirve cada botón**. Está pensado para que cualquiera que lo lea entienda el simulador de eventos discretos sin conocerlo de antes.

1. ¿Qué simula el programa?

Una **playa de estacionamiento** con N sectores (8 por defecto). Los autos llegan, estacionan un rato y al final **pagan en una zona de cobro** antes de irse. Es una **simulación de eventos discretos (DES)**: el reloj salta de evento en evento (no avanza segundo a segundo), y en cada salto se actualiza el estado del sistema en una fila del **vector de estado**.

El objetivo del TP es responder: **a)** la recaudación, **b)** cuánto recaudaría con 10 lugares, y **c)** el porcentaje de utilización de la playa.

2. Arquitectura: tres archivos

Archivo	Responsabilidad
simulacion.py	El motor : variables aleatorias, eventos, lógica de la playa, cola de cobro, estadísticas. No sabe nada de la interfaz.
euler.py	La integración numérica (método de Euler) que calcula el tiempo de cobro. Separado a propósito (ver decisión 4).
main.py	La interfaz gráfica (PyQt5): parámetros, botones, tabla del vector de estado, pestaña de auditoría Euler, estadísticas.

La idea (buena práctica) es separar **lógica** de **interfaz**: el motor se puede probar y verificar solo, sin abrir ventanas.

3. Cómo funciona el motor (DES)

Objetos

- **Playa**: N lugares (Libre / Ocupado).
- **Zona de cobro**: capacidad 1 (Libre / Ocupado).
- **Auto**: con 3 atributos clave — **estado**, **importe a pagar** y **tipo** (Pequeño/Grande/Utilitario).

Eventos

- **Llegada de auto**: entra si hay lugar; si está lleno, sigue de largo y no vuelve.
- **Fin de estacionamiento**: va a cobrar (si la zona está libre) o espera en la cola de cobro.
- **Fin de cobro**: paga, se va y libera la zona de cobro para el siguiente.

El loop principal

Hay una **cola de eventos** (un *heap*) ordenada por tiempo. El loop saca el próximo evento, mueve el reloj a ese instante, ejecuta lo que corresponde y guarda una fila. Repite hasta que se cumple una condición de corte.

4. Decisiones de diseño (el corazón del TP)

1. El tiempo se mide en horas

Qué hicimos: Todo el reloj y los tiempos están en horas (las llegadas, las duraciones de 1–4 h, etc.).

Por qué: Porque el estacionamiento se mide en horas y así coincide directo con nuestro Excel del TP.

2. Llegadas con distribución exponencial

Qué hicimos: El tiempo entre llegadas se genera con $t = -\text{media} \cdot \ln(1 - \text{RND})$, con $\text{media} = 13 \text{ min} = 13/60 \text{ h}$.

Por qué: El enunciado da un 'índice entre llegadas de 13'. Elegimos exponencial negativa (lo habitual para llegadas aleatorias) y lo dejamos documentado.

3. El RND se trunca a 2 decimales

Qué hicimos: El número aleatorio se corta a 2 decimales (queda entre 0.00 y 0.99) antes de usarlo.

Por qué: Dos motivos: (1) que el RND que se muestra sea exactamente el que se usó, para poder recalcular la fila a mano (tabla auditable); (2) al quedar en $[0.00, 0.99]$, nunca pasa que $1 - \text{RND} = 0$, evitando el error $\log(0)$.

4. El tiempo de cobro se calcula por integración numérica (Euler), en un archivo aparte

Qué hicimos: El cobro no tiene una fórmula cerrada: se obtiene integrando $dD/dt = C + 0.2 \cdot T + t^2$ con el método de Euler, hasta que D llega al umbral. Esa lógica vive en `euler.py`.

Por qué: Es la 'parte continua' que pide el TP5. Lo separamos en su propio archivo para que se entienda y se pueda auditar por separado (la pestaña 'Auditoría Euler' muestra esa tabla).

5. El umbral del cobro depende del tipo de auto (por eso el auto guarda su tipo)

Qué hicimos: La integración corta cuando $D = 180$ (autos Grandes) o 130 (Pequeños y Utilitarios). Cada auto guarda su **tipo** desde que llega.

Por qué: El tipo se sortea al llegar, pero el cobro ocurre **después** (al terminar de estacionar). Si no guardáramos el tipo en el objeto auto, al cobrar no sabríamos qué umbral usar.

6. C = autos en la cola de cobro, SIN contar al que entra

Qué hicimos: Cuando un auto empieza a cobrar, C es la cantidad de autos que quedan esperando en la cola (los demás), no el que está cobrando. Ej.: si esperaban 3 y entra 1, $C = 2$.

Por qué: Es la lectura literal del enunciado ('autos esperando el cobro al inicio del mismo'): el que entra ya no espera. Lo verificamos contra nuestro Excel (caso real: 2 esperando, entra 1, $C = 1$).

7. El auto libera su lugar apenas termina de estacionar (no queda 'bloqueado')

Qué hicimos: Al ocurrir `fin_estacionamiento`, el auto deja libre su sector **de inmediato** y pasa a la zona/cola de pago. Si la zona de cobro esta ocupada, espera en la cola **sin ocupar ningún sector**.

Por qué: Si retuviera el lugar mientras espera el cobro, habría que inventar un estado 'bloqueado' y el % de utilización contaría tiempo en que el auto ya no está estacionado. Liberándolo enseguida, la ocupación mide solo autos realmente estacionados (pregunta c).

8. El paso de Euler h se mantiene en 0.1 como mínimo

Qué hicimos: El parámetro h es configurable, pero usamos 0.1.

Por qué: Con $h = 0.1$ la tabla de integración queda en ~80–115 filas (legible para mostrarla). El error frente al valor exacto es de apenas ~3 a 9 segundos por cobro: despreciable.

9. La tabla de Euler muestra solo t , D y dD/dt

Qué hicimos: Sacamos las columnas $t(i+1)$ / $D(i+1)$ y la columna de paso h por fila.

Por qué: Esas columnas eran redundantes (el $i+1$ es la fila siguiente) y la h es un parámetro fijo que va arriba, no repetido en cada fila. Queda más limpio y fácil de leer.

10. RND sembrados que reproducen el Excel, con fallback a aleatorio

Qué hicimos: Al principio el programa usa los mismos RND de nuestro Excel del TP; cuando se agotan, genera aleatorios. Se controla con `USAR_SEMILLA`.

Por qué: Sirve para **verificar**: la simulación da exactamente las mismas filas que la planilla hecha a mano. Una vez validado, se pasa a aleatorio para correr días enteros.

11. La cantidad de filas (i) limita la simulación, no solo la vista

Qué hicimos: Si pedís mostrar i filas desde la hora j , el motor **simula solo esa ventana** y se detiene; no simula un millón de filas para mostrar 20.

Por qué: Evita trabajo inútil y hace la app más rápida cuando solo querés inspeccionar un tramo. (Para estadísticas de un día completo, dejá i vacío y poné el tiempo máximo.)

12. Tabla con 'modelo virtual' y columnas congeladas

Qué hicimos: La tabla solo dibuja las celdas visibles (modelo virtual) y mantiene fijas las 3 primeras columnas ($\#$, Reloj, Evento) al hacer scroll horizontal.

Por qué: Es un requisito de la cátedra: miles de filas sin que la pantalla 'parpadee', con los datos de referencia siempre a la vista. Además el scroll vertical de las columnas fijas va sincronizado con el resto.

5. La interfaz: parámetros y botones

Parámetros (todos editables)

Parámetro	Qué controla
Lugares (N)	Cantidad de sectores de la playa (8; usar 10 para la pregunta b).
Media llegada (min)	Promedio de tiempo entre llegadas de autos (13').
Tiempo máx (h)	Hasta qué hora simular (X). Corta la simulación.
T (Euler)	Constante T de la ecuación del cobro.
h (Euler)	Paso de integración de Euler (0.1).
Precio Pequeño / Grande / Utilitario	Tarifa por hora de cada tipo de auto.
% Pequeño / Grande / Utilitario	Probabilidad de cada tipo de auto (45 / 25 / 30).
% 1h / 2h / 3h / 4h	Probabilidad de cada duración de estacionamiento (50 / 30 / 15 / 5).
Mostrar desde hora j	Desde qué hora arranca la ventana que se muestra/simula.

Cant. filas i	Cuántas filas mostrar (y hasta dónde simular). Vacío/0 = sin límite.
---------------	--

Botones

Botón	Qué hace
■ Simular	Corre la simulación con los parámetros actuales (en un hilo aparte para que la ventana no se congele) y llena el vector de estado, la última fila y las estadísticas.
Re-aplicar i/j	Vuelve a aplicar el filtro de visualización (mostrar i filas desde la hora j) sobre lo ya simulado, sin volver a simular.
Copiar a Excel	Copia el vector de estado visible al portapapeles en formato tabulado, listo para pegar en Excel o Google Sheets.

Pestañas y zonas

- **Vector de Estado:** la tabla principal, fila por fila de la simulación. Debajo, la **última fila** (estado final del sistema).
- **Auditoría Euler:** la tabla de cada integración del cobro (columnas t, D, dD/dt), identificada por auto, tipo, C y umbral.
- **Barra inferior:** las estadísticas en vivo (recaudación, % utilización, % abandono, cantidad de filas e integraciones).

6. Cómo correrlo

```
pip install PyQt5
cd TP5-Playa
python main.py
```

Para verificar el motor sin interfaz se pueden importar simulacion y euler desde un script y leer `vector_estado` y `estadisticas()`.